

UNIVERSITY OF MORATUWA

Department of Electronic and Telecommunication Engineering

EN3150 – Pattern Recognition

Assignment 03

Flower Image Classification

Convolutional Neural Networks and Transfer Learning

Submission type:	Group report
Group members:	230145E – Dilhan W.A. 230375H – Lenmini B.L.W. 230070T – Balasooriya B.R.B.D. 230211E – Gunsekara K.S.
Module:	EN3150 – Pattern Recognition
Dataset:	TF Flowers (3,670 RGB images)
Repository:	GitHub project page

October 2026

Contents

1 Introduction **1**

2 Dataset and experimental design **1**

2.1 Dataset 1

2.2 Reproducible data split 1

2.3 Training-only augmentation 2

3 Methods **2**

3.1 Custom CNN architectures 2

3.2 Optimizer comparison 2

3.3 Custom-model training and selection 3

3.4 Transfer learning 3

4 Results **3**

4.1 Optimizer selection 3

4.2 Custom architecture comparison 4

4.3 Pretrained model comparison 6

5 Discussion **9**

6 Conclusion **10**

List of Figures

1 Training and validation loss and accuracy for the main optimizer comparison. . . 3

2 Validation behaviour across the tested SGD momentum values. 4

3 Model A training and validation curves. 5

4 Model A test confusion matrix. 5

5 Model B training and validation curves. 6

6 Model B test confusion matrix. 6

7 MobileNetV2 training and validation curves; the figure marks the fine-tuning stage and best validation epoch. 7

8 MobileNetV2 test confusion matrix. 8

9 EfficientNet-B0 training and validation curves. 8

10 EfficientNet-B0 test confusion matrix. 9

Abstract

This project studies five-class flower recognition on the TF Flowers dataset. The work prepares a reproducible stratified data split, compares optimizers using a compact depthwise separable convolutional neural network, evaluates a standard CNN against a parameter-efficient CNN, and fine-tunes two ImageNet-pretrained mobile architectures [2, 3]. The saved experiment records show that the standard custom CNN achieved 79.82% test accuracy, while the compact custom model achieved 76.18%. MobileNetV2 reached the highest recorded test accuracy (85.09%), at substantially higher model size and computational cost than the compact model. These results illustrate the accuracy–efficiency trade-off between training a small model from scratch and adapting pretrained visual features.

Keywords: image classification, convolutional neural network, TF Flowers, depthwise separable convolution, transfer learning

1 Introduction

Image classification assigns an input image to one label from a fixed set of classes. Flower recognition is a useful small-scale vision task because different species can have similar colour and texture, while examples within a class vary in pose, scale, background, and lighting. A convolutional neural network (CNN) learns spatial filters that can represent increasingly complex visual patterns.

This report documents the implementation and recorded results for five flower classes: dandelion, daisy, tulips, sunflowers, and roses. It covers dataset preparation, optimizer selection, custom architecture design, held-out test evaluation, and fine-tuning of MobileNetV2 and EfficientNet-B0. All numerical results below are taken from the repository's committed result files; training was not repeated while preparing this report.

2 Dataset and experimental design

2.1 Dataset

The project uses the TF Flowers dataset distributed through TensorFlow Datasets [1], containing 3,670 labelled RGB images in five classes. Images are directly resized to 64×64 pixels by bilinear interpolation with antialiasing. Direct resizing is simple and gives a fixed input shape, but it can change the original aspect ratio.

2.2 Reproducible data split

The preparation script uses seed 42 and performs a separate random shuffle within each class before assigning approximately 70% to training, 15% to validation, and the remainder to testing. The script checks that every source image occurs exactly once in the combined split. Integer rounding yields the counts in table 1; the saved split manifest records the assignment of each source example.

Table 1: Class counts in the prepared data splits.

Class	Training	Validation	Test	Total
Dandelion	629	135	134	898
Daisy	443	95	95	633
Tulips	559	120	120	799
Sunflowers	489	105	105	699
Roses	449	96	96	641
Total	2,569	551	550	3,670

Only training data is shuffled for model fitting. Custom CNN inputs are converted to float values in $[0, 1]$. Pretrained-model inputs remain in $[0, 255]$ until the architecture-specific preprocessing layer is applied. The validation set guides model selection. The test set is reserved for final evaluation in the Q4 and Q5 scripts.

2.3 Training-only augmentation

For custom-model and pretrained-model training, the repository applies random horizontal flips, rotations with a factor of 0.1, and zoom with a factor of 0.1 to training batches only. In Keras, the 0.1 rotation factor corresponds to up to approximately $\pm 36°$; this follows the implementation in `train_eval.py`, whose descriptive docstring summarizes the rotation more loosely. Validation and test examples are not augmented.

3 Methods

3.1 Custom CNN architectures

Both custom models accept $64 \times 64 \times 3$ inputs and use four feature blocks with output channel counts 32, 64, 128, and 256. Each block ends with 2×2 max pooling, reducing the spatial resolution from 64 to 4 pixels. Global average pooling feeds dropout with rate 0.3, a 128-unit ReLU dense layer, and a five-unit softmax classifier.

Model A (standard CNN) uses a 3×3 Conv2D layer, batch normalization, and ReLU in each block. Convolution biases are disabled because the following batch-normalization layer provides a learned offset.

Model B (depthwise separable CNN) replaces each standard convolution with a 3×3 depthwise convolution followed by a 1×1 pointwise convolution. Each operation is followed by batch normalization and ReLU6. This factorization reduces multiply–accumulate operations (MACs) and parameter count while retaining the same overall block widths and classifier head.

3.2 Optimizer comparison

The optimizer experiment trains Model B for 20 epochs with batch size 32 and sparse categorical cross-entropy. Runs use the same seed, initial weights, data split, and shuffled batch order. The compared optimizers are Adam at 10^{-3}, SGD at 10^{-2}, SGD with momentum 0.9 at 10^{-2}, and a momentum sweep at 0.5 and 0.99. The validation set is used to compare runs; the saved script keeps the test split unused for this step.

3.3 Custom-model training and selection

The Q4 experiment trains each custom model for 30 epochs with Adam, learning rate 10^{-3}, batch size 32, and sparse categorical cross-entropy. Training uses the same split and data augmentation. A ReduceLROnPlateau schedule is enabled after the initial epochs, with a minimum learning rate of 10^{-5}; checkpoints retain the weights with the best validation accuracy. The test metrics are computed after selection. The recorded comparison reports accuracy, macro precision, macro recall, parameter counts, MACs, model size, and median epoch time.

3.4 Transfer learning

MobileNetV2 and EfficientNet-B0 are initialized with ImageNet weights. The 64×64 inputs are bilinearly upsampled to 128×128 inside each model. Training is two-stage: ten epochs train a new five-class head while the backbone is frozen, then ten epochs fine-tune the top 40 backbone layers at learning rate 10^{-5}. The head-only learning rate is 10^{-3}. Batch normalization layers in the pretrained backbone remain in inference mode. The best validation-accuracy checkpoint across both stages is used for final test evaluation.

4 Results

4.1 Optimizer selection

Table 2 reports the best validation accuracy for each saved run. Adam reached the highest value, 73.68% at epoch 20, and was selected for the custom-model comparison. SGD with momentum 0.9 was the strongest SGD variant at 72.41%. The additional momentum sweep did not outperform Adam.

Table 2: Optimizer comparison on Model B (validation set).

Optimizer	Learning rate	Best validation accuracy	Best epoch
Adam	10^{-3}	0.7368	20
SGD	10^{-2}	0.6987	16
SGD, momentum 0.9	10^{-2}	0.7241	19
SGD, momentum 0.5	10^{-2}	0.7132	18
SGD, momentum 0.99	10^{-2}	0.6606	18

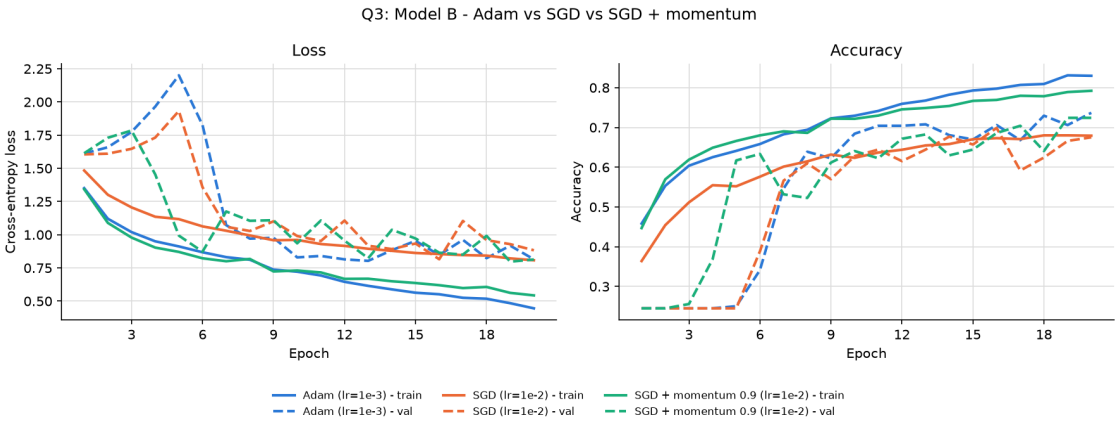


Figure 1: Training and validation loss and accuracy for the main optimizer comparison.

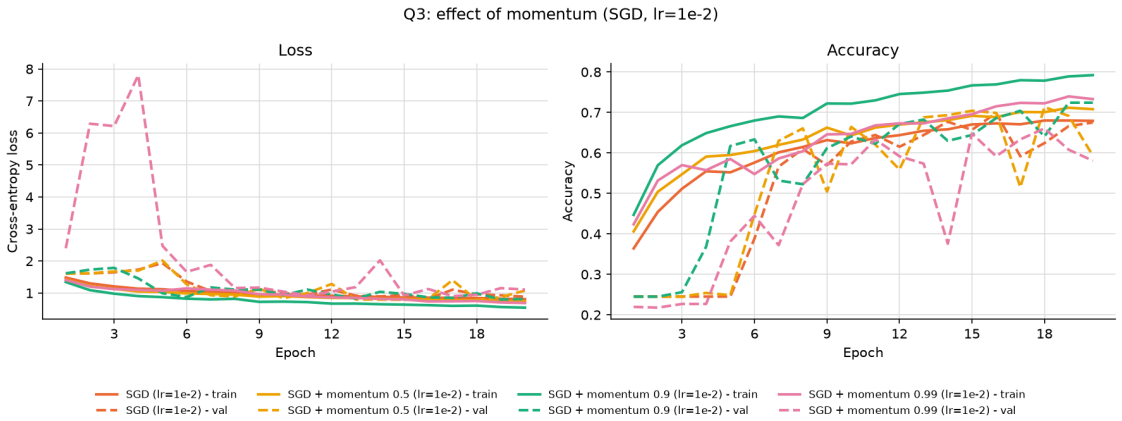


Figure 2: Validation behaviour across the tested SGD momentum values.

4.2 Custom architecture comparison

Model A produced the higher test accuracy (79.82% versus 76.18%) and higher macro precision and recall. Model B reduced trainable parameters by about 81% and MACs by about 88% relative to Model A. Its serialized model file was about one quarter the size. Median epoch times were similar in the recorded environment, so the principal observed advantage was lower model and compute footprint rather than a large wall-clock speedup.

Table 3: Custom CNN performance and computational footprint.

Metric	Model A	Model B
Trainable parameters	422,437	80,102
Non-trainable parameters	960	1,414
Total parameters	423,397	81,516
MACs per image	60.20 M	7.34 M
Model size on disk	1.69 MiB	0.42 MiB
Median time per epoch	10.2 s	9.3 s
Best validation accuracy	79.85% (epoch 21)	74.59% (epoch 25)
Test accuracy	79.82%	76.18%
Macro precision	81.05%	75.99%
Macro recall	78.97%	75.73%

The saved class-level metrics show differences by category. Model A has its highest recall on sunflowers (89.5%) and dandelions (88.8%), while daisy recall is lowest at 65.3%. Model B reaches 91.4% recall on sunflowers but 64.6% on roses. These per-class values suggest that aggregate accuracy conceals different class-specific error patterns.

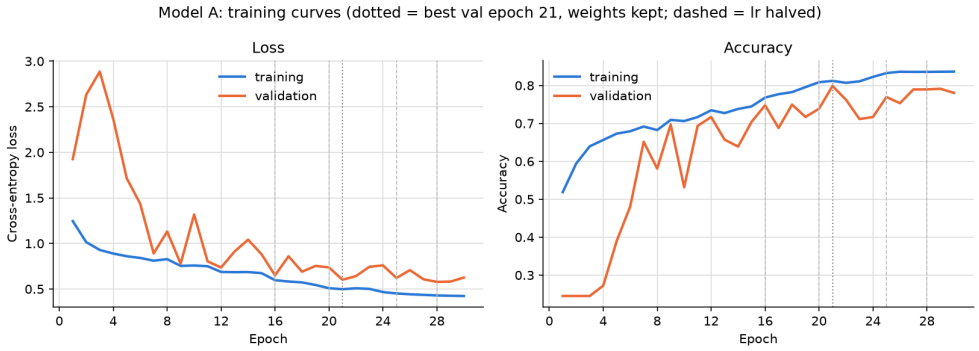


Figure 3: Model A training and validation curves.

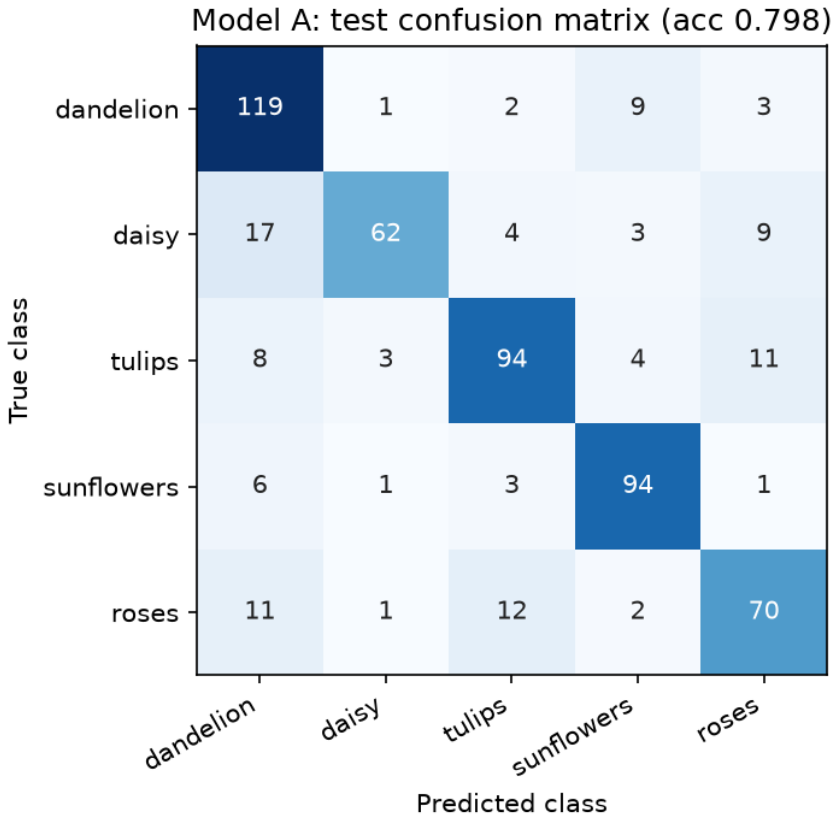


Figure 4: Model A test confusion matrix.

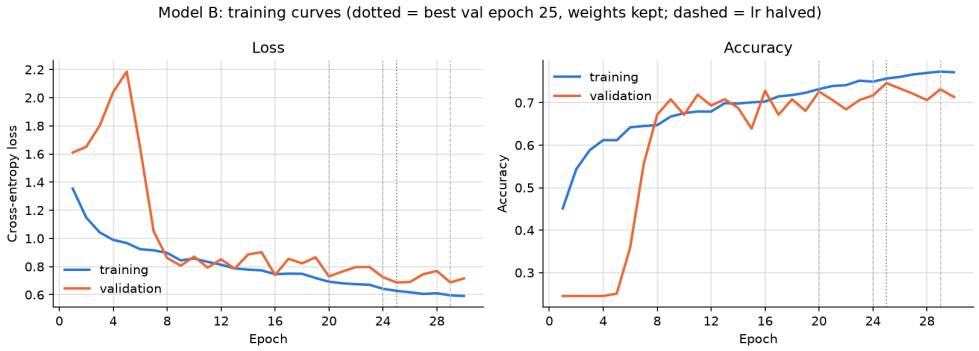


Figure 5: Model B training and validation curves.

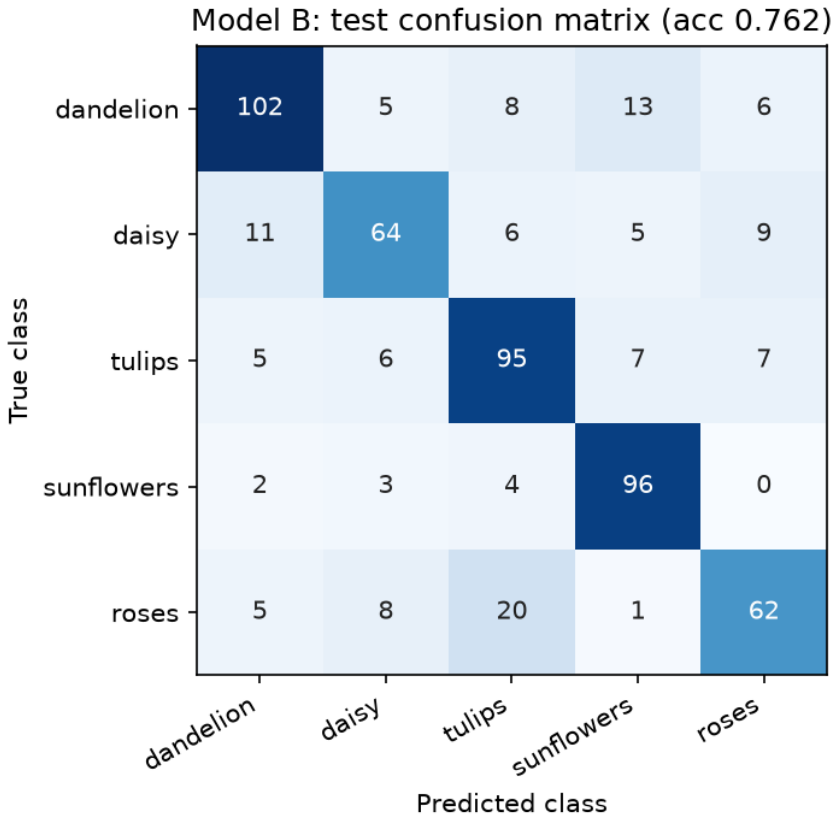


Figure 6: Model B test confusion matrix.

4.3 Pretrained model comparison

MobileNetV2 achieved the highest recorded test accuracy (85.09%) and macro precision (85.72%) and recall (85.12%). EfficientNet-B0 scored 83.82% test accuracy. MobileNetV2 also had fewer parameters, lower MACs, and a smaller saved file than EfficientNet-B0 in this setup. Both pretrained models exceed the custom Model B accuracy, but Model B remains substantially smaller and less computationally expensive.

Table 4: Transfer-learning results on the held-out test split.

Metric	MobileNetV2	EfficientNet-B0
Total parameters	2,264,389	4,055,976
Model size on disk	9.20 MB	16.28 MB
MACs per image	97.8 M	126.0 M
Input to backbone	128×128	128×128
Best validation accuracy	86.03% (epoch 16)	84.57% (epoch 20)
Test accuracy	85.09%	83.82%
Macro precision	85.72%	84.07%
Macro recall	85.12%	83.44%
Median epoch time, head stage	18.7 s	34.3 s
Median epoch time, fine-tune stage	22.0 s	36.6 s

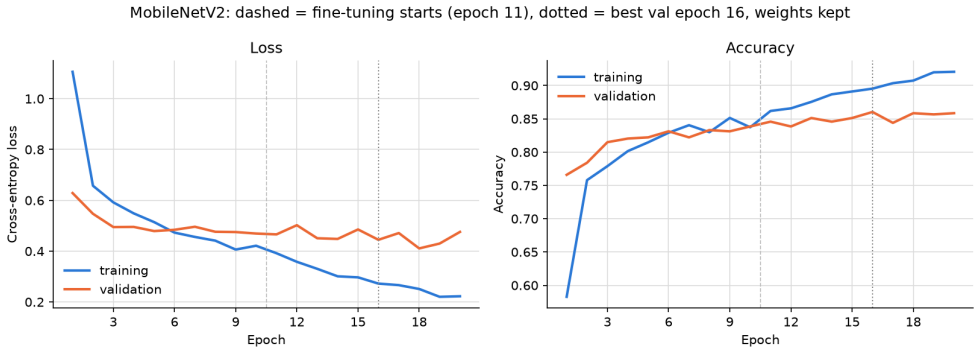


Figure 7: MobileNetV2 training and validation curves; the figure marks the fine-tuning stage and best validation epoch.

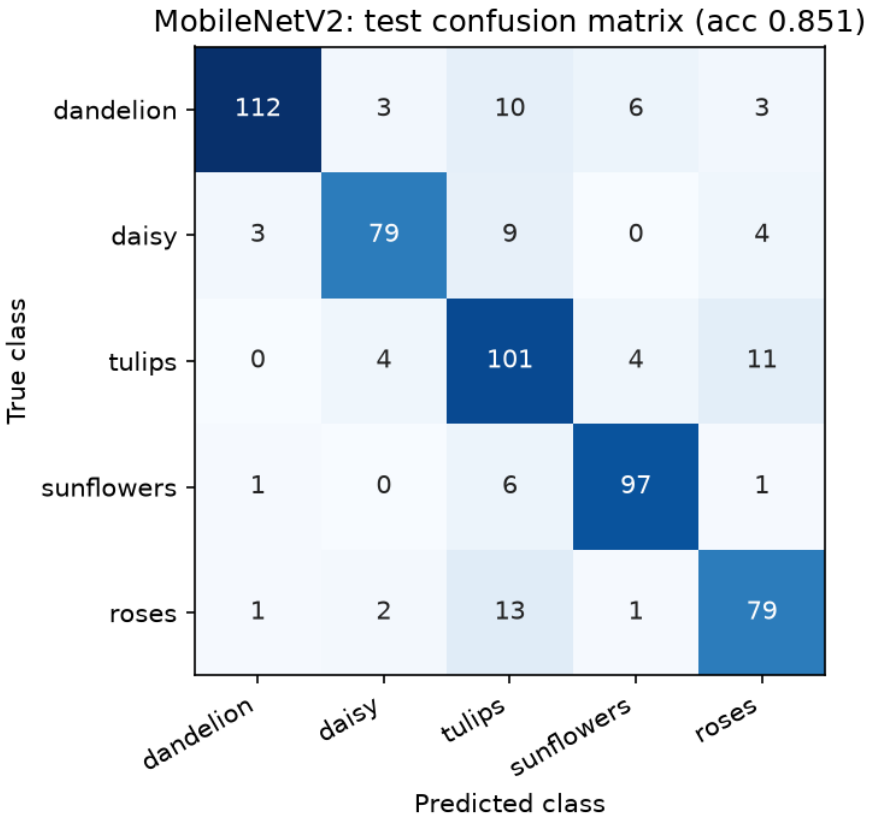


Figure 8: MobileNetV2 test confusion matrix.

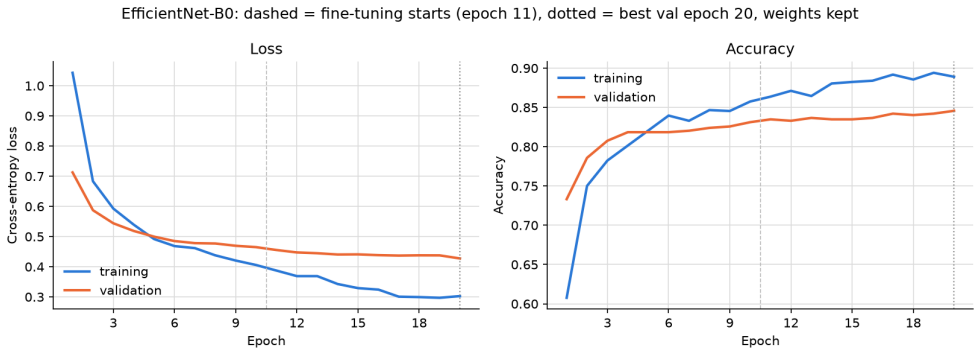


Figure 9: EfficientNet-B0 training and validation curves.

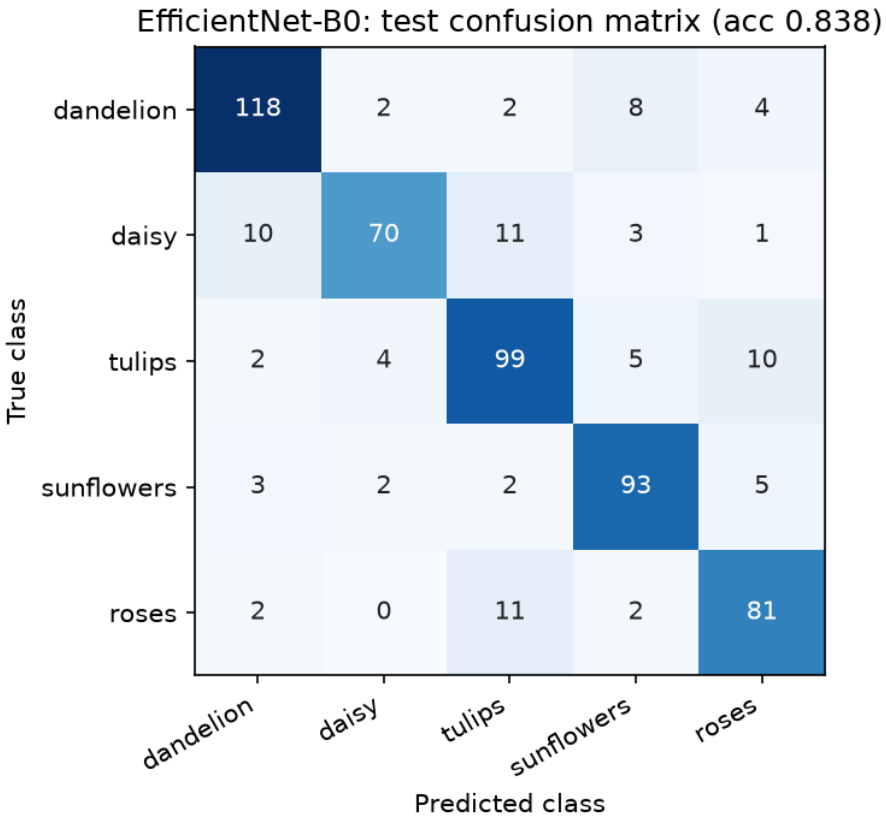


Figure 10: EfficientNet-B0 test confusion matrix.

5 Discussion

The results support two practical observations. First, the standard custom CNN was more accurate than the depthwise separable custom CNN, while the latter used far fewer parameters and MACs. This is a useful trade-off when deployment resources matter. Second, transfer learning improved test accuracy on this dataset: MobileNetV2 gained 5.27 percentage points over Model A and 8.91 points over Model B. These gains came with approximately 27.8 times the parameters and 13.3 times the MACs compared with Model B.

MobileNetV2 is the strongest option when maximizing the recorded accuracy is the priority. Model B is more suitable when model size and inference cost are the dominant constraints. EfficientNet-B0 did not surpass MobileNetV2 here and required more parameters, MACs, model storage, and epoch time in the saved experiment.

The comparison has limitations. Only one fixed stratified split and one set of training runs are reported; no repeated-seed confidence intervals are available. The timing values depend on the unrecorded execution hardware and should be interpreted only as within-run comparisons. Upsampling a 64-pixel image to 128 pixels does not add image detail. The repository also does not record a per-class evaluation for the pretrained models in the summary tables, although full confusion matrices and class metrics are saved. Finally, the image resizing step may distort aspect ratios.

6 Conclusion

The project implemented a reproducible flower-classification pipeline and compared optimizer choices, custom CNN architectures, and pretrained backbones. Adam provided the best validation result in the optimizer sweep. Among custom models, Model A achieved 79.82% test accuracy, while Model B reduced computation and parameter count substantially with a modest accuracy drop. MobileNetV2 achieved the best recorded test accuracy at 85.09%, at a larger computational and storage cost. The results demonstrate that the preferred model depends on whether predictive performance or lightweight deployment is the main requirement.

Reproducibility and repository

The source code, saved metric files, plots, and serialized models are included in the project repository. The scripts are `prepare_data.py`, `train_optimizers.py`, `models.py`, `train_eval.py`, and `train_sota.py`. The repository README documents the required Python packages and run order. The results in this report are transcribed from `results/q3`, `results/q4`, and `results/q5`.

References

[1] TensorFlow Datasets, "tf_flowers," dataset catalogue. https://www.tensorflow.org/datasets/catalog/tf_flowers.

[2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018.

[3] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," *Proceedings of the 36th International Conference on Machine Learning*, 2019.

[4] University of Moratuwa, "The Emblem of University." https://uom.lk/emblem-university.php.